

AIDATACENTER — DEMO

Live bootstrap demo

Rev 2026-07-18

Generated from Markdown — edit the .md source and regenerate (`python3 docs/md2pdf.py`)

Contents

1	Run	3
2	The bootstrap process	4
2.1	Phase -1 — NetBox source of truth	4
2.2	Phase 0 — Seed host	4
2.3	Phase 1 — Switch provisioning (ZTP)	4
2.4	Phase 2 — Metal3 + CAPI control plane	4
2.5	Phase 3 — GPU workers	5
2.6	Phase 4 — Flux platform	5
2.7	Phase 5 — Model deploy (Kimi K2 Thinking)	5
2.8	Phase 6 — API online (gateway · scheduler · first tokens)	5
2.9	Phase 7 — Acceptance	5
3	Views	7
4	Controls	8

Browser simulator for the 64× B200 AIDataCenter bring-up path. No hardware or cluster required.

1 Run

```
python3 -m http.server 8765 --directory demo  
# → http://127.0.0.1:8765
```

Or open `index.html` directly in a browser.

2 The bootstrap process

The demo walks the same ordered bring-up defined in `docs/bootstrap.md`. Each phase below lists its goal, the real commands it simulates, and its exit criteria.

2.1 Phase —1 — NetBox source of truth

Goal: every device, IP, and cable exists in NetBox before anything boots.

```
cd bootstrap/netbox && docker compose up -d
export NETBOX_URL=http://127.0.0.1:8081
export NETBOX_TOKEN=$(bash scripts/create_token.sh)
python3 scripts/import_seed.py          # seed/site.yaml → racks, devices, interfaces, cables
bash ../scripts/netbox-sync.sh          # → inventory/cluster.yaml + ipam.yaml + dhcp-hosts.conf
```

Demo: NetBox containers start, seed import reports device/cable counts (8× B200 server, 8× 7060DX5-32 leaf, 2× 7060DX5-64S spine, 2× 7010TX-48 OOB, 169 cables), inventory export is written. **Exit:** inventory snapshot generated; all devices planned.

2.2 Phase 0 — Seed host

Goal: seed01 serves DHCP/DNS/NTP/iPXE so everything else can boot from the network.

```
sudo bash scripts/00-seed-host.sh          # dnsmasq + chrony + matchbox from NetBox export
curl -sfL https://get.k3s.io | sh -        # mgmt cluster for CAPI
```

Demo: seed01 powers on in the BOOT rack; terminal renders dnsmasq.conf (14 static leases), confirms mgmt 10.10.0.0/24 and OOB 10.20.0.0/24 plans. **Exit:** DHCP answers for all inventory hosts; iPXE chainloads via matchbox.

2.3 Phase 1 — Switch provisioning (ZTP)

Goal: the fabric comes up before servers: OOB first (BMC reachability), then spines, then the 8 rail leaves — cabling verified against NetBox.

```
ZTP via DHCP option 67 → EOS image + startup-config per device
oob-sw1/2 (7010TX-48) → spine1/2 (7060DX5-64S) → leaf-rail0..7 (7060DX5-32)
```

Demo: view auto-switches to the **cabling map** — brown OOB cables pulse as the 7010TXs come up, orange leaf↔spine uplinks connect as BGP establishes, teal host↔leaf mesh stays dark until workers exist. Terminal shows EOS boot, `show lldp neighbors`, and BGP Established. **Exit:** 12/12 switches ready; 64 leaf↔spine uplinks up; BMC plane reachable; EVPN/VXLAN overlay established (16 sessions, vrf **storage** + vrf **edge**, border to campus).

2.4 Phase 2 — Metal3 + CAPI control plane

Goal: 3 control-plane nodes (+ 3 utility) provisioned as bare metal via Redfish/Ironic.

```
bash scripts/01-install-capi-metal3.sh          # clusterctl init --infrastructure metal3
BMC_USERNAME=... BMC_PASSWORD=... bash scripts/02-apply-cluster.sh
clusterctl describe cluster ai-cluster -n metal3 # wait Ready
```

Demo: each node walks registering → inspecting → provisioning → provisioned; API VIP 10.10.0.20:6443 announced when kubeadm initializes. **Exit:** 3 CP Ready; VIP serves 6443.

2.5 Phase 3 — GPU workers

Goal: all 8 workers provisioned, racked 4-per GPU rack, one NIC per rail.

```
kubectl apply -f capi/workers-gpu.yaml # MachineDeployment gpu-workers x8
```

Demo: workers provision rack-by-rack (GPU-1 then GPU-2); each host↔leaf DAC cable in the map connects as its worker finishes; rail strip flows green. Nodes carry `nvidia.com/gpu=true:NoSchedule` until the GPU Operator lands. **Exit:** 8 workers Ready; 8× ConnectX-7 rails link-up per node.

2.6 Phase 4 — Flux platform

Goal: GitOps brings up the platform in dependency order.

```
GIT_URL=https://git.example.com/org/super.git bash scripts/03-install-flux.sh
```

Demo: the Platform panel reconciles in order — `platform-core` → `security` → `netbox` / `registry` / `observability` / `bus` → `gpu` → `serving`. When `platform-gpu` goes Ready, `nvidia.com/gpu` capacity becomes 8×8 = 64. **Exit:** 8 base kustomizations Ready (`platform-models` / `platform-api` follow in phases 5–6).

2.7 Phase 5 — Model deploy (Kimi K2 Thinking)

Goal: stage 594 GB of INT4 weights onto the hot FS and bring up 4 TP=8 replicas (`docs/serving.md` §1–2).

```
kubectl -n inference get job kimi-k2-thinking-download -w # hf download → CHECKSUMS → `current`
kubectl -n inference get isvc kimi-k2-thinking -w # vLLM TP=8, 1 replica per XE9680
```

Demo: download progress at the §6.4 hot-tier rate, checksum verification, atomic current promote; replicas 1–4 load to HBM on worker01–04 (workers 05–08 stay the Ray/research pool). The **Model** stat flips – → `staged` → `K2` ×4 TP8. **Exit:** `inferenceservice/kimi-k2-thinking` Ready – 4/4 replicas (32 GPUs).

2.8 Phase 6 — API online (gateway · scheduler · first tokens)

Goal: external OpenAI-compatible access at `https://inference.ai.local/v1` through the EVPN edge VRF (`docs/overlay.md` §4, `docs/serving.md` §3–4).

```
flux reconcile kustomization platform-api --with-source
curl -s https://inference.ai.local/v1/models -H "x-api-key: ***"
```

Demo: MetalLB announces VIP 10.50.0.10 to leaf-rail0/7 (vrf edge), Envoy gateway terminates TLS with API-key auth + per-key quotas, the EPP inference scheduler comes up with 4 endpoints, and a live chat completion streams token-by-token (reasoning + answer) with the EPP routing decision and the archive record landing in Redpanda. **Exit:** first tokens served via the VIP; request captured to the 1-year archive.

2.9 Phase 7 — Acceptance

```
bash scripts/04-smoke.sh
```

Demo: terminal prints the acceptance checklist from `docs/bootstrap.md` §9 plus the overlay/serving gates: CP Ready, 8/8 workers, 64 GPUs, DCGM scraped, Harbor push/pull, Vault raft = 3, Redpanda reachable, EVPN

sessions/VTEPs, 4/4 replicas + balanced EPP spread, auth 401/429 behavior, archive parquet ≤ 60 s. **Exit:**
ACCEPTANCE PASS – 64× B200 online · Kimi K2 Thinking serving at <https://inference.ai.local/v1>.

3 Views

- **Racks & fabric** — 4 rack elevations (GPU-1/GPU-2/BOOT/STOR) + rail strip; status dots pulse per device
- **Cabling map** — all **169 cables** from docs/cabling.md, lighting up as endpoints come online:
 - 64× host↔leaf DAC 400G (teal) · workerN.rail{i} → leaf-rail{i}.Ethernet{N}
 - 64× leaf↔spine AOC 400G (orange) · 8 uplinks/leaf, 4 per spine
 - 41× OOB/mgmt (brown) · BMC (VLAN 20) + mgmt0 (VLAN 10) + switch Ma1 (ZTP) + MLAG peer
 - Live per-class counters; hover any cable for its NetBox label, click any node to inspect
- **Live terminal** — streams the operator session: `docker compose up`, `import_seed.py`, ZTP/EOS boot + EVPN overlay checks, `clusterctl`, Metal3 BMH transitions, Flux reconciles, the Kimi K2 weight staging, a token-streamed chat completion, and `04-smoke.sh` PASS lines (typewriter effect, speed-aware, clearable)
- **Event log** — compact phase/state transitions
- **Inspector** — click any device or Flux app for details + actions

4 Controls

- **Run all** — auto-play the full pipeline (auto-switches to the cabling view during switch bring-up)
- **Step** — advance one checkpoint at a time
- **Pause / Reset** — freeze or wipe state
- **Speed** — multiplier on simulated delays (also affects terminal typing)
- Inspector actions: provision now · reboot · inject fail · force Flux reconcile

Inventory, topology, and cable plan match `bootstrap/netbox/seed/site.yaml`, `docs/bootstrap.md`, and `docs/cabling.md`.